

LEROY AI // ARCHITECTURE & VECTOR RAG TECHNICAL GUIDE

How Embeddings, Knowledge Base Data Collection & Supabase pgvector Work

1. End-to-End System Execution Flow

When an executive types a query into Leroy AI (e.g. *'Which sales rep generated the highest ARR?'*), the request travels through a 4-step **LangGraph StateGraph Execution Pipeline**:

1. **Intent Router Node**: Classifies query into EMPLOYEE_HR, REVENUE_OPTIMIZATION, or GREETING.
2. **Vector Retriever Node**: Converts query text into a 384-dimensional dense vector and executes Cosine Similarity search over Supabase pgvector.
3. **Telemetry Analytics Node**: Aggregates real-time financial metrics (\$4.85M ARR, \$404K MRR, 118.5% NRR).
4. **Groq LLM Generator Node**: Formulates structured Bloomberg-style executive markdown using Groq LLaMA-3.3-70B.

2. How Vector Embeddings Are Generated

Vector embeddings convert unstructured text (sentences, paragraphs, PDFs, employee bios) into a 384-dimensional dense float vector array where semantically similar concepts sit close together in vector space.

- **Model Used**: SentenceTransformer('all-MiniLM-L6-v2')
- **Output Vector Dimensions**: 384 floats per text chunk (e.g. [-0.0245, 0.0812, 0.0394, ...])
- **Speed & Latency**: ~5ms per chunk.

```
# Python Code (backend/db.py) from sentence_transformers import SentenceTransformer model =  
SentenceTransformer('all-MiniLM-L6-v2') def generate_embedding(text: str) -> list[float]: return  
model.encode(text).tolist() # Returns 384-d vector
```

3. How Knowledge Base Data is Collected & Fed into RAG

Leroy AI collects and vectorizes data through **three automated pipelines**:

- **Pipeline 1: Integrated Employee Platform** — Creating or updating employees auto-generates a profile summary vector doc (role, salary, \$ ARR impact, skills, bio) inserted into Supabase.
- **Pipeline 2: Direct Document Upload API** — Uploading strategy playbooks via /api/knowledge converts text into 384-d vector chunks.
- **Pipeline 3: Corporate PDF Ingestion** — Corporate strategy PDFs (ReportLab/PyPDF) are parsed into chunks and embedded into Supabase knowledge_documents table.

4. How Things Work in Supabase pgvector

Supabase operates on **PostgreSQL** with the official **pgvector extension** enabled.

1. **Vector Column**: knowledge_documents table includes an embedding vector(384) column.
2. **IVFFlat Indexing**: Fast cosine distance search (USING ivfflat (embedding vector_cosine_ops)).
3. **Cosine Similarity Search**: Stored function match_documents(query_embedding, match_count) calculates similarity score as 1 - (embedding <=> query_embedding).
4. **Row Level Security (RLS)**: Disabled on dev tables (ALTER TABLE ... DISABLE ROW LEVEL SECURITY) to allow anon API access.

5. Leroy AI Core Concepts Matrix

Concept	Implementation
LLM Engine	Groq API (LLaMA-3.3-70B-Versatile)
Agentic Pipeline	LangGraph StateGraph Workflow (Router -> Retriever -> Analytics -> Gen)
Embedding Model	SentenceTransformer ('all-MiniLM-L6-v2') 384-D
Vector Database	Supabase PostgreSQL + pgvector extension
Similarity Operator	Cosine Similarity via <=> operator (1 - distance)
UI Design System	Bloomberg Terminal theme (React + Vite + JetBrains Mono)